FIAP · Global Solution 2026.2 · CitizenTech · Engenharia de Software 4ES

Caminho Livre: Deploy Blindado

Secure DevOps Tools & Cloud Computing · Prof. Gabriel Henrique Boyczuk Arcanjo

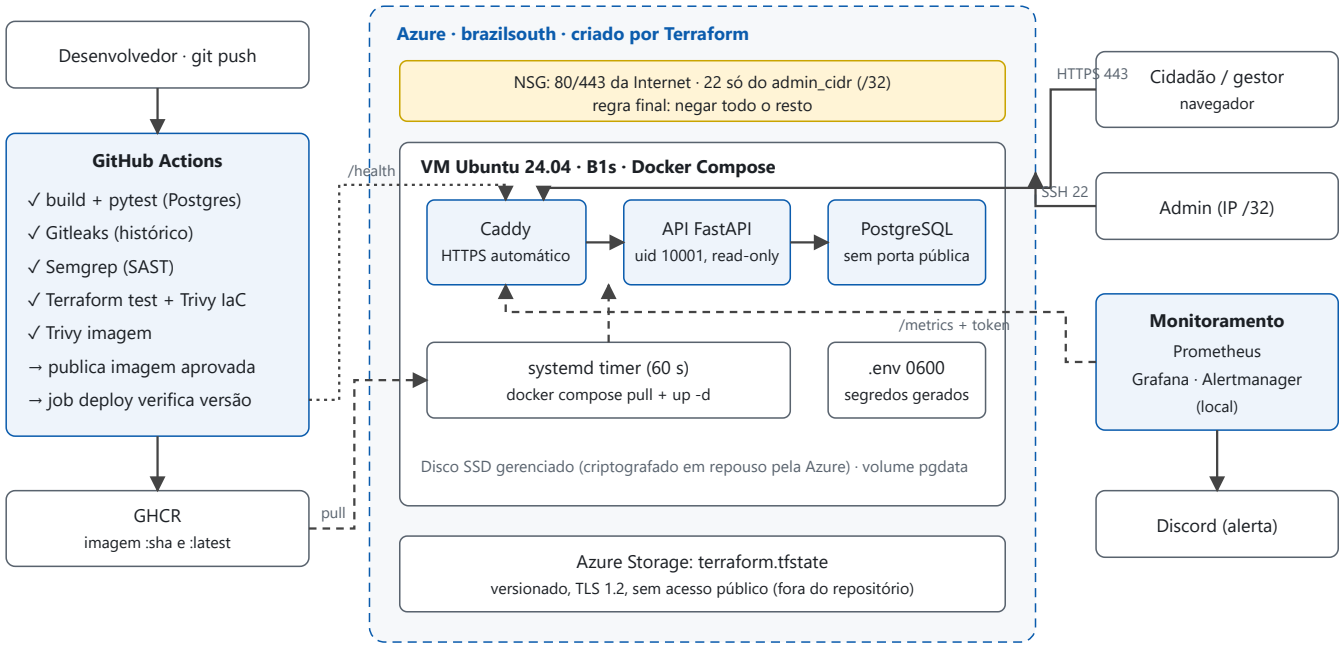
Integrante (nome completo)	RM
preencher	preencher
preencher	preencher
preencher	preencher
preencher	preencher
preencher	preencher

Link	Endereço
MVP publicado (URL pública)	https://caminholivre-gs.brazilsouth.cloudapp.azure.com (confirmar com `terraform output app_url`)
Repositório público	https://github.com/OWNER/caminho-livre
Pipeline (GitHub Actions)	https://github.com/OWNER/caminho-livre/actions

1. Ficha Técnica do MVP

Nome do produto	Caminho Livre
Cidadão e território	Pedestres com deficiência ou mobilidade reduzida, idosos e cuidadores. O piloto cobre o entorno do campus FIAP Paulista (São Paulo, SP).
Problema, em uma frase	Rampas bloqueadas, calçadas obstruídas e travessias com barreira são relatadas de forma dispersa e ninguém acompanha se foram resolvidas.
Proposta de valor	O cidadão registra a barreira no mapa e acompanha o status. O gestor do local recebe uma fila auditável para revisar e encerrar cada relato.
Arquitetura: componentes	Página web (HTML + Leaflet), API FastAPI, PostgreSQL e Caddy (HTTPS) em Docker Compose, numa VM Azure criada por Terraform. CI/CD com GitHub Actions. Prometheus, Grafana e Alertmanager.
Fontes de dados	OpenStreetMap (mapa base e malha viária) e relatos do próprio MVP. Os pontos de demonstração são sintéticos e vêm identificados na tela.
Modelos de IA utilizados	Detector de obstáculos customizado (YOLO), do componente Applied Computer Vision. Fica fora do escopo desta entrega.
URL do MVP e repositório	Ver tabela de links acima.
Indicadores de impacto	Relatos recebidos e revisados, tempo até a primeira revisão e taxa de resolução confirmada. São indicadores de processo, não prova de melhoria física.
ODS conectados	ODS 10 (redução das desigualdades) e ODS 11 (cidades e comunidades sustentáveis).

2. Arquitetura



Fluxo: cada `git push` passa por 5 portões (testes, Gitleaks, Semgrep, Trivy IaC e Trivy imagem). Só a imagem aprovada na `main` recebe a tag `latest` no GHCR. Na VM, um `systemd timer` aplica a imagem nova em até 60 s (deploy *pull-based*), e o job `deploy` do pipeline só passa quando `/health` responde com o SHA do commit. O CI não guarda credencial da Azure e o SSH não fica aberto para a internet.

Por que VM (IaaS) e não PaaS: o custo é previsível e cabe no free tier de estudante. A stack é a mesma que roda local (`docker-compose.yml`) e o Terraform descreve toda a rede. O preço disso é que patches do SO, backup e disponibilidade ficam com a equipe (seção 7).

3. Container e Infraestrutura como Código (5,0)

3.1 Dockerfile enxuto, não-root, sem segredo e stack em docker compose

Requisito	Como foi atendido	Arquivo
Enxuto	Multi-stage: o `pip` roda no estágio de build e só os pacotes instalados vão para a imagem `python:3.12-slim`. `.dockerignore` exclui testes e cache.	`app/Dockerfile`
Usuário não-root	`USER 10001`. No compose: `read_only`, `cap_drop: ALL`, `no-new-privileges`, `mem_limit 256m`.	`docker-compose.yml`
Sem segredo na imagem	Os segredos só entram por variável de ambiente em runtime. O Trivy roda com `--scanners secret` na imagem a cada push.	workflow
Patches do SO na imagem	`apt-get upgrade` no estágio final. Foi uma correção exigida pelo Trivy (seção 4.3).	`app/Dockerfile`
Stack com docker compose	`db` (Postgres sem porta publicada) + `api` + `proxy` (Caddy). O mesmo arquivo roda local e na VM, e o override local só acrescenta o `build`.	`docker-compose.yml`

PRINT PENDENTE: salve a captura como docs/evidencias/01-compose-ps.png e gere o PDF de novo

Figura 1: `docker compose ps` com os 3 serviços saudáveis e `docker compose exec api id` mostrando uid 10001 (não-root).

3.2 Terraform: state e .tfvars fora do repositório, porta admin nunca em 0.0.0.0/0

Requisito	Como foi atendido
Nuvem criada por código	`infra/terraform/main.tf`: resource group, VNet/subnet, NSG, IP público com DNS, NIC e VM Ubuntu 24.04. A VM é provisionada por cloud-init: Docker, stack, `.env` 0600, swap, timer de deploy e `unattended-upgrades`.
State fora do repositório	Backend `azurerm` num Storage Account com versionamento, TLS 1.2 e sem acesso público, criado por `infra/bootstrap/criar-state.sh`. O `backend.hcl`, o `*.tfstate` e o `*.tfvars` estão no `.gitignore`; o repositório só tem os `*.example`.
Porta admin nunca 0.0.0.0/0	A regra SSH usa `var.admin_cidr`, e a *validation* só aceita prefixos de /24 a /32 (0.0.0.0/0 é recusado no `plan`). O `terraform test` prova os dois bloqueios no CI, com providers simulados. Os únicos acessos públicos são 80/443, e a última regra nega todo o resto.
Sem senha digitada	Senha do Postgres, token admin e token de métricas vêm de `random_password`. O SSH aceita só chave; login por senha está desabilitado.

```
$ terraform test
  run "bloqueia_ssh_aberto_para_a_internet"... pass
  run "bloqueia_faixa_larga_no_ssh"... pass
  run "infra_segura_com_ip_do_admin"... pass
Success! 3 passed, 0 failed.
```

PRINT PENDENTE: salve a captura como docs/evidencias/02-terraform-apply.png e gere o PDF de novo

Figura 2: `terraform apply` concluído com os outputs (`app_url`, `public_ip`).

PRINT PENDENTE: salve a captura como docs/evidencias/03-nsg-portal.png e gere o PDF de novo

Figura 3: regras de entrada do NSG no portal Azure. SSH só aceita o IP /32 do administrador.

PRINT PENDENTE: salve a captura como docs/evidencias/04-tfstate-storage.png e gere o PDF de novo

Figura 4: `caminho-livre.tfstate` no container `tfstate` do Storage Account (fora do Git).

4. Pipeline DevSecOps (4,0)

Arquivo: `.github/workflows/devsecops.yml`. Roda em **todo push**, em qualquer branch.

Job	Ferramenta	Bloqueia quando
Build e testes	pytest + Postgres 16 (service)	qualquer teste falha (fluxo cidadão/gestor, validação de território, 401 sem token, LGPD)
Gitleaks	Gitleaks 8.28 (histórico completo)	algum segredo aparece em qualquer commit
Semgrep	p/default, p/python, p/dockerfile, p/secrets	algum achado aparece (`--error`)
Terraform + Trivy IaC	fmt, validate, test + `trivy config`	formatação, validação, teste de segurança ou misconfiguração HIGH/CRITICAL
Imagem + Trivy	`trivy image --scanners vuln,secret`	há CVE HIGH/CRITICAL com correção ou segredo na imagem
Deploy automático	job com environment `producao`	o SHA não entra no ar em 5 min, ou `/metrics` responde sem token

Segredos no GitHub Secrets: `METRICS_TOKEN` é usado no smoke test autenticado. O push para o GHCR usa o `GITHUB_TOKEN` efêmero, com permissão `packages: write` só no job de imagem; o padrão do workflow é `contents: read`. Nenhuma credencial da Azure fica no CI.

PRINT PENDENTE: salve a captura como docs/evidencias/05-github-secrets.png e gere o PDF de novo

Figura 5: Settings → Secrets and variables → Actions (`METRICS_TOKEN` e a variável `APP_URL`).

PRINT PENDENTE: salve a captura como docs/evidencias/06-pipeline-verde.png e gere o PDF de novo

Figura 6: execução completa na `main` com todos os jobs verdes, incluindo o deploy.

PRINT PENDENTE: salve a captura como docs/evidencias/07-deploy-log.png e gere o PDF de novo

Figura 7: log do job *Deploy automático* confirmando a nova versão (SHA) no ar.

4.3 Execução barrada e corrigida

Caso real no desenvolvimento: na primeira varredura da imagem, o Trivy barrou o build com 2 vulnerabilidades HIGH no pacote `zlib1g` da imagem base Debian 13 (CVE-2026-85091, *heap buffer overflow*), já corrigidas na versão `1:1.3.dfsg+really1.3.1-1+deb13u1`. A correção foi aplicar os patches do SO no Dockerfile (`apt-get upgrade`). Com isso a varredura passou a mostrar 0 HIGH/CRITICAL.

Demonstração no GitHub Actions: numa branch descartável foi commitado um segredo falso (`DATABASE_PASSWORD = "..."`) e o Gitleaks barrou o pipeline. A correção foi remover o segredo *do histórico* (`git reset` + `push --force`), não só apagar o arquivo, porque o Gitleaks varre todos os commits. Num caso real, o segredo também seria rotacionado.

PRINT PENDENTE: salve a captura como docs/evidencias/08-pipeline-barrado.png e gere o PDF de novo

Figura 8: execução barrada pelo Gitleaks (segredo falso detectado).

PRINT PENDENTE: salve a captura como docs/evidencias/09-pipeline-corrigido.png e gere o PDF de novo

Figura 9: mesma branch, execução corrigida com todos os portões verdes.

5. Observabilidade (3,0)

A API expõe `/metrics` (Prometheus) protegido por token Bearer. Prometheus, Grafana e Alertmanager rodam localmente e raspam o MVP publicado via HTTPS, como o enunciado permite. Os rótulos de rota usam o *template* (`/api/admin/reports/{relato_id}`), o que evita cardinalidade alta e IDs nas métricas.

Sinal de ouro	**Métrica / PromQL**
Latência	`histogram_quantile(0.95, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))` (p50/p95/p99)
Tráfego	`sum by (route) (rate(http_requests_total[1m]))`
Erros	`sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))`
Saturação	`process_resident_memory_bytes` (limite de 256 MB), `rate(process_cpu_seconds_total[1m])` , `http_requests_in_progress`

Alerta	**Condição**	**Severidade**
CaminhoLivreFora	`up == 0` por 1 min	critical
TaxaDeErrosAlta	> 5% de respostas 5xx por 1 min	critical
LatenciaP95Alta	p95 > 1 s por 5 min	warning
SaturacaoMemoria	> 200 MB por 5 min	warning

Como o alerta foi disparado: com o banco parado na VM (`docker compose stop db`) e o site em uso, a API passou a responder 500 e o `TaxaDeErrosAlta` foi para *firing*. O Alertmanager enviou a notificação ao Discord. Com `docker compose start db` , o alerta foi resolvido e a notificação de *resolved* também foi enviada. É uma falha real de dependência, sem nenhum endpoint de "erro proposital" exposto em produção.

PRINT PENDENTE: salve a captura como docs/evidencias/10-grafana-sinais-de-ouro.png e gere o PDF de novo

Figura 10: dashboard *Caminho Livre - 4 sinais de ouro* no Grafana, alimentado pelo MVP publicado.

PRINT PENDENTE: salve a captura como docs/evidencias/11-alerta-disparado.png e gere o PDF de novo

Figura 11: alerta `TaxaDeErrosAlta` em *firing* no Prometheus/Alertmanager e a mensagem recebida no Discord.

6. MVP publicado

PRINT PENDENTE: salve a captura como docs/evidencias/12-mvp-url-publica.png e gere o PDF de novo

Figura 12: MVP aberto na URL pública (HTTPS, janela anônima), com mapa, formulário e lista de relatos.

7. Responsabilidade compartilhada (modelo IaaS)

Camada	Microsoft Azure	Equipe Caminho Livre
Física e virtualização	Datacenter, hardware, rede física, hipervisor, isolamento entre clientes	—
Rede	Infraestrutura da VNet, mecanismo do NSG, IP público	Regras do NSG (Terraform): só 80/443 públicos, SSH só do admin, negar o resto
Armazenamento	Durabilidade do disco gerenciado e criptografia em repouso (chaves da plataforma)	Backup lógico do Postgres (pendente, ver riscos), proteção do Storage do tfstate
Sistema operacional	Imagem Ubuntu publicada pela Canonical no marketplace	Patches (`unattended-upgrades`), hardening, chave SSH sem senha
Containers e dependências	—	Imagem base atualizada, CVEs (Trivy), usuário não-root, capabilities
Aplicação	—	Código (testes, Semgrep), autenticação do gestor, CSP, validação de entrada
Identidade e segredos	Entra ID / RBAC da assinatura	Segredos gerados pelo Terraform, `.env` 0600, GitHub Secrets, rotação
Dados e LGPD	Região escolhida (brazilsouth) e certificações da plataforma	Minimização, sanitização, retenção e resposta a titulares (controladora)
Monitoramento	Saúde do host físico	Métricas da aplicação, alertas e resposta a incidente

8. LGPD: onde ficam os dados pessoais

Dado	É pessoal?	Onde fica	Proteção
Conta de cidadão	—	Não existe	Minimização: o piloto não tem cadastro
Coordenada do relato	Pode indicar onde a pessoa estava	Postgres (volume `pgdata`) na VM, disco em brazilsouth	Arredondada para 4 casas (~11 m) antes de gravar
Descrição livre	Pode conter contatos	Postgres (`descricao_sanitizada`)	E-mail, telefone e CPF trocados por `[removido]` antes de gravar (testado)
Endereço IP do visitante	Sim	Não é armazenado	Caddy sem access log, API sem log de corpo, métricas sem IP
Fotos	Podem ter rostos/placas	Não são armazenadas neste MVP	O módulo de Visão processa sob demanda (fora desta entrega)
Segredos (tokens, senha do banco)	Não pessoais, mas sensíveis	`.env` 0600 na VM, tfstate remoto, GitHub Secrets	Nunca no Git (Gitleaks); o tfstate tem acesso restrito à assinatura

Em trânsito, tudo passa por HTTPS (certificado Let's Encrypt emitido pelo Caddy). Em repouso, o disco gerenciado da Azure é criptografado pela plataforma. Os dados ficam no Brasil (região `brazilsouth`).

9. Estimativa de custo

Fonte: Azure Retail Prices API (preços públicos pay-as-you-go em USD, região `brazilsouth`), consultada em 11/11/2026. O mês considerado tem 730 h.

Recurso	Preço unitário	USD/mês
VM Standard_B1s (Linux)	US$ 0,0168/h	12,26
IP público IPv4 Standard estático	US$ 0,005/h	3,65
Disco do SO Standard SSD E4 (32 GB)	US$ 4,48/mês	4,48
Storage do tfstate (poucos KB)	—	< 0,01
Saída de dados (primeiros 100 GB/mês gratuitos)	—	0,00
GitHub Actions e GHCR (repositório público)	—	0,00
Prometheus/Grafana/Alertmanager (local)	—	0,00
Total sem benefícios		**≈ 20,40**
Com o benefício de 750 h/mês de B1s das contas Azure gratuitas/estudante (12 meses, confirmar na assinatura)		≈ 8,14

Para comparação, a mesma VM em `eastus` custa US$ 0,0104/h (≈ US$ 7,59/mês). Mesmo assim a escolha foi `brazilsouth`, para manter os dados no Brasil e a latência baixa para o cidadão de São Paulo. Para escalar, os próximos passos seriam uma VM B2s e um PostgreSQL gerenciado com backup automático. Esses itens não foram orçados aqui.

10. Riscos conhecidos e próximos passos

- **Backup do banco:** ainda não automatizado. Próximo passo: `pg_dump` diário para um Storage Account com retenção, ou migrar para o Azure Database for PostgreSQL.
- **Ponto único de falha:** uma VM só. Aceitável no piloto; o monitoramento avisa (`CaminhoLivreFora`) quando ela cai.
- **Monitoramento local:** só alerta enquanto a máquina da equipe está ligada. Próximo passo: Azure Monitor ou Grafana Cloud.
- **Rate limiting:** o POST de relatos ainda não tem limite por IP. Próximo passo: `rate_limit` no Caddy.

11. Checklist da rubrica

Critério	Pts	Evidência
Dockerfile enxuto, não-root, sem segredo e docker compose	2,0	Seção 3.1, Fig. 1, `app/Dockerfile`, `docker-compose.yml`
Terraform, state/.tfvars fora do repo, admin nunca 0.0.0.0/0	3,0	Seção 3.2, `terraform test`, Figs. 2–4, `.gitignore`
Actions: build, Gitleaks, Semgrep, Trivy por push; Secrets; deploy automático; barrada e corrigida	4,0	Seção 4, Figs. 5–9
Dashboard com os 4 sinais de ouro e um alerta disparado	3,0	Seção 5, Figs. 10–11
Arquitetura, responsabilidade compartilhada, LGPD, custo, README	2,0	Seções 2, 7, 8 e 9; README com Ficha Técnica

12. Conclusão

O Caminho Livre chega ao cidadão por um caminho em que nenhum passo é manual. A infraestrutura é descrita em Terraform e testada antes de existir. Cada alteração de código passa por testes e por quatro varreduras de segurança, e só a imagem aprovada é publicada e aplicada automaticamente. Depois de publicado, o produto é observado pelos quatro sinais de ouro, com alertas que já foram disparados por uma falha real. Os riscos que ainda existem, principalmente backup e redundância, estão listados com o próximo passo de cada um.